

Sliding Subarray Beauty – Full Revision Note (Interview Ready)

1 Warm-up Question (to build intuition)

Problem

Given numbers in range $[-2, 2]$, find the **smallest** or **x-th smallest** number present using a frequency array.

Why this warm-up?

Because **Sliding Subarray Beauty** is nothing but: - maintaining a frequency array - repeatedly finding the x-th smallest element

2 Why Frequency Array? (First Big Why)

- Numbers are in a **fixed small range** (e.g. `[-50, 50]`)
- Sorting every window is too slow
- Frequency array gives $O(1)$ insert & delete

Mapping Logic (MOST IMPORTANT)

Array indices cannot be negative, so we **shift values**.

If values are in `[-50, 50]`:

```
offset = 50
index = value + offset
value = index - offset
```

This mapping is **reversible**, which is why it always works.

3 First Smallest Number Logic (Simplest Case)

Goal

Find the **smallest number present**.

Code

```
for (int i = 0; i < freq.length; i++) {  
    if (freq[i] > 0) {  
        answer = i - offset;  
        break;  
    }  
}
```

Why this works

- We scan from **smallest value** → **largest value**
- First bucket with `freq > 0` means:
 - number exists
 - it is the smallest present

Key Rule

Smallest does NOT mean smallest possible value, it means smallest **present** value.

4 X-th Smallest Number Logic (Core Concept)

Goal

Find the **x-th smallest** number in current window.

Code

```
int count = 0;  
for (int i = 0; i < freq.length; i++) {  
    count += freq[i];  
    if (count >= x) {  
        answer = i - offset;  
        break;  
    }  
}
```

Why this works

- `freq[i]` = how many times value `(i - offset)` occurs
- `count` = how many numbers we have seen so far
- When `count >= x`, we have crossed the x-th smallest element

Example Dry Run

```
values present = [-2, 0, 0, 1]
x = 2
```

i	value	freq[i]	count
0	-2	1	1
1	-1	0	1
2	0	2	3 ← count >= 2 → answer = 0

5 Actual Problem: Sliding Subarray Beauty

Problem Statement

You are given: - an integer array `nums` - window size `k` - integer `x`

For each subarray of size `k`, return the **x-th smallest negative number**. If fewer than `x` negative numbers exist → return `0`.

6 Why Sliding Window? (Second Big Why)

- Each subarray is **fixed size k**
- Windows overlap
- Instead of rebuilding from scratch:
 - remove left element
 - add right element

This keeps time complexity optimal.

7 Why We Remove Elements (CRITICAL DOUBT)

Code Line

```
freq[nums[start] + offset]--;
```

Why remove?

Because the left element **leaves the window**.

If we don't remove: - frequency array becomes incorrect - old elements remain counted - x-th smallest logic breaks

Interview Line

"When the window slides forward, the leftmost element exits the window, so I decrement its frequency to keep the data accurate."

8 Full Algorithm (Step-by-Step)

1. Initialize frequency array of size 101
 2. Build frequency for first window
 3. For each window:
 4. find x-th smallest using cumulative count
 5. store result
 6. remove left element
 7. add right element
-

9 Full Dry Run (Small Example)

```
nums = [-1, -2, -3, 4]
k = 3
x = 2
```

First window [-1, -2, -3]

Sorted: [-3, -2, -1] 2nd smallest = -2

Slide window

Remove -1, add 4 Window = [-2, -3, 4] Sorted negatives = [-3, -2] 2nd smallest = -2

10 Final Interview Explanation (Say This)

"Since the numbers lie in a fixed range, I use a frequency array instead of sorting. I maintain this array using a sliding window by adding the incoming element and removing the outgoing one. To find the x-th smallest element, I scan the frequency array cumulatively until the count reaches x. I use an offset to map negative values to valid indices."

1 1 Common Mistakes to Avoid

- ❌ Forgetting to remove left element
 - ❌ Confusing index with value
 - ❌ Thinking smallest means smallest possible
 - ❌ Not explaining offset logic clearly
-

1 2 Confidence Check

If you understand: - why `i - offset` works - why we decrement freq on slide - why `count >= x` gives x-th smallest

👤 You are **interview ready** for this problem.

✅ This problem is hard. Your doubts were VALID and CORRECT. Most candidates never reach this clarity.

Revisit this note before interviews.